TOKEN MANAGEMENT & AI COST ARCHITECTURE

Implementation Guide
Production Code · 2026

Python 3.12	FastAPI	Redis Stack	LiteLLM	OpenTelemetry	LangGraph	Grafana	Kafka

TABLE OF CONTENTS

1

MODEL ROUTING

Complexity Classifier · Routing Engine · LiteLLM Config

Architecture Overview

The routing layer scores every incoming request with a lightweight Sentence-BERT classifier (~80 MB, <10ms inference) and dispatches to the cheapest model tier capable of handling it. RouteLLM (Stanford) demonstrates 50% cost reduction at 95% quality parity. The classifier runs before any LLM call — its own cost is negligible (~$0.00001 per classification).

Tier Decision Matrix

Tier	Models	Cost/1K tok	Complexity Score	Use Cases
Nano	claude-haiku-4-5, gemini-flash, gpt-4o-mini	$0.00025–$0.0006	< 0.30	Autocomplete, syntax Q&A;, formatting, classification
Mid	claude-sonnet-4-6, gpt-4o	$0.003–$0.005	0.30–0.70	Chat, debugging, code review, multi-file context
Frontier	claude-opus-4-6, o3	$0.015–$0.075	> 0.70	Architecture design, complex reasoning, security analysis

1a — Complexity Classifier (classifier.py)

Uses cosine similarity against anchor embeddings for "trivial" vs "complex" reference prompts. Length and code-depth heuristics add to the base score. Runs in-process — no network hop.

```python
# router/classifier.py
from sentence_transformers import SentenceTransformer, util
import torch

model = SentenceTransformer("all-MiniLM-L6-v2")  # 80MB, <10ms inference

SIMPLE_ANCHORS = [
    "what does this function do",
    "fix this syntax error",
    "autocomplete the next line",
]
COMPLEX_ANCHORS = [
    "design a distributed system architecture",
    "debug this race condition across microservices",
    "refactor this 2000-line module with SOLID principles",
]

simple_embs = model.encode(SIMPLE_ANCHORS, convert_to_tensor=True)
complex_embs = model.encode(COMPLEX_ANCHORS, convert_to_tensor=True)

def score_complexity(prompt: str) -> float:
    """Returns 0.0 (trivial) to 1.0 (frontier-worthy)."""
    emb = model.encode(prompt, convert_to_tensor=True)
    sim_simple = float(util.cos_sim(emb, simple_embs).max())
    sim_complex = float(util.cos_sim(emb, complex_embs).max())

    token_count = len(prompt.split())
    length_boost = min(token_count / 2000, 0.30)
    depth_boost = min(prompt.count("\n") / 200, 0.20)

    base = (sim_complex - sim_simple + 1) / 2  # normalize [0, 1]
    return min(1.0, base + length_boost + depth_boost)

def select_model(score: float, explicit_tier: str | None = None,
                 budget_remaining_pct: float = 1.0) -> str:
    if explicit_tier:
        return {"nano": "claude-haiku-4-5",
                "mid": "claude-sonnet-4-6",
                "frontier": "claude-opus-4-6"}[explicit_tier]
    # Budget pressure: downgrade thresholds when budget low
    if budget_remaining_pct < 0.10: return "claude-haiku-4-5"
    if budget_remaining_pct < 0.20:
        return "claude-haiku-4-5" if score < 0.7 else "claude-sonnet-4-6"
    if score < 0.30: return "claude-haiku-4-5"
    if score < 0.70: return "claude-sonnet-4-6"
    return "claude-opus-4-6"
```

1b — Routing Engine (FastAPI, main.py)

Single endpoint all consumers call. Pipeline: cache check → classify → budget check → LiteLLM dispatch → record usage → cache store. All steps tagged for observability.

```python
# router/main.py
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
from classifier import score_complexity, select_model
from cache import semantic_cache_get, semantic_cache_set
from budget import get_budget_status, record_usage
from litellm import completion
import time, uuid, structlog

app = FastAPI()
log = structlog.get_logger()

class RoutedRequest(BaseModel):
    messages:       list[dict]
    max_tokens:     int | None = None
    team_id:        str
    feature_id:     str
    use_case:       str
    explicit_tier:  str | None = None
    environment:    str = "production"

@app.post("/v1/chat/completions")
async def routed_completion(req: RoutedRequest):
    request_id   = str(uuid.uuid4())[:8]
    start        = time.monotonic()
    user_prompt  = next(
        (m["content"] for m in reversed(req.messages) if m["role"] == "user"), ""
    )

    # 1. Cache check (zero LLM cost if hit)
    if cache_hit := semantic_cache_get(user_prompt):
        return {**cache_hit, "cached": True, "cost_usd": 0.0}

    # 2. Classify complexity
    score = score_complexity(user_prompt)

    # 3. Budget check
    budget = get_budget_status(req.team_id, req.feature_id)
    if budget["remaining_pct"] == 0:
        raise HTTPException(429, detail={"error": "token_budget_exhausted"})

    # 4. Model selection
    model   = select_model(score, req.explicit_tier, budget["remaining_pct"])
    max_tok = req.max_tokens or _default_max(req.use_case)

    # 5. LLM call via LiteLLM (provider-agnostic)
    response = completion(model=model, messages=req.messages, max_tokens=max_tok,
                          metadata={"team_id": req.team_id,
                                    "feature_id": req.feature_id,
                                    "complexity": score})
    # 6. Record & cache
    record_usage(req.team_id, req.feature_id, response.usage.total_tokens, model)
    semantic_cache_set(user_prompt, response, use_case=req.use_case)

    log.info("routed", model=model, complexity=round(score,2),
             tokens=response.usage.total_tokens,
             latency_ms=int((time.monotonic()-start)*1000))
    return response

def _default_max(use_case: str) -> int:
    return {"autocomplete":64,"chat":512,"review":2048,"architecture":4096}.get(use_case,512)
```

1c — LiteLLM Provider Config (litellm_config.yaml)

YAML

```yaml
# litellm_config.yaml  (mount as Kubernetes ConfigMap)
model_list:
  - model_name: nano
    litellm_params:
      model: anthropic/claude-haiku-4-5
      api_key: os.environ/ANTHROPIC_API_KEY
  - model_name: nano-fallback
    litellm_params:
      model: gemini/gemini-flash-2-0
      api_key: os.environ/GOOGLE_API_KEY
  - model_name: mid
    litellm_params:
      model: anthropic/claude-sonnet-4-6
      api_key: os.environ/ANTHROPIC_API_KEY
  - model_name: frontier
    litellm_params:
      model: anthropic/claude-opus-4-6
      api_key: os.environ/ANTHROPIC_API_KEY

router_settings:
  routing_strategy: least-busy
  retry_policy:
    num_retries: 3
  fallbacks:
    - [nano, nano-fallback]

litellm_settings:
  success_callback: ["langfuse"]
  cache: true
  cache_params:
    type: redis
    host: redis-cache.internal
    port: 6379
```

2 SEMANTIC CACHE

Redis Vector Search · TTL Strategy · Cache Warming

Semantic caching embeds the user query, finds similar past queries via cosine similarity (threshold 0.92), and returns the cached response — zero LLM cost. Production impact: 30–60% LLM call reduction. Embedding cost is ~$0.00002 per query — 100x cheaper than the LLM call it avoids. Uses RedisStack (includes RediSearch + Vector module).

TTL Strategy by Content Type

Content Type	TTL	Threshold	Cache?	Reason
Syntax Q&A; (stable knowledge)	30 days	0.93	Yes	High repetition, stable answers
Framework docs questions	7 days	0.92	Yes	Docs change slowly
Architecture patterns	24 hours	0.90	Yes	Moderate reuse
Code review (generic style)	4 hours	0.93	Selective	Avoid specific PR context
Security guidance	1 hour	0.95	Yes, strict	Must be current; high precision
Debugging help	2 hours	0.91	Selective	Context-specific
PII-containing prompts	Never	N/A	NO — never	Privacy violation risk
Customer data context	Never	N/A	NO — never	Data isolation requirement

2a — Semantic Cache Implementation (cache.py)

```python
# cache/semantic_cache.py
import redis, json, hashlib, time, re
from openai import OpenAI
from redis.commands.search.field import VectorField, TextField
from redis.commands.search.indexDefinition import IndexDefinition, IndexType
from redis.commands.search.query import Query
import numpy as np

r    = redis.Redis(host="redis-cache.internal", port=6379, decode_responses=False)
oai  = OpenAI()
SIMILARITY_THRESHOLD  = 0.92
EMBEDDING_MODEL       = "text-embedding-3-small"    # $0.00002 / 1K tokens
CACHE_INDEX           = "semantic_cache_idx"

TTL_MAP = {
    "syntax_qa": 30*86400, "framework_docs": 7*86400,
    "architecture": 86400, "debugging": 2*3600,
    "security": 3600,      "code_review": 4*3600,
    "test_generation": 86400, "default": 4*3600,
}

def _embed(text: str) -> list[float]:
    return oai.embeddings.create(input=text, model=EMBEDDING_MODEL).data[0].embedding

def ensure_index():
    try:
        r.ft(CACHE_INDEX).info()
    except Exception:
        schema = (
            TextField("$.prompt",      as_name="prompt"),
            TextField("$.use_case",    as_name="use_case"),
            VectorField("$.embedding","HNSW",
                        {"TYPE":"FLOAT32","DIM":1536,
                         "DISTANCE_METRIC":"COSINE"}, as_name="embedding"),
        )
        r.ft(CACHE_INDEX).create_index(
            schema,
            definition=IndexDefinition(prefix=["semcache:"],
                                       index_type=IndexType.JSON))

def semantic_cache_get(prompt: str) -> dict | None:
    ensure_index()
    emb_bytes = np.array(_embed(prompt), dtype=np.float32).tobytes()
    q = (Query("*=>[KNN 3 @embedding $vec AS score]")
         .sort_by("score").return_fields("response","score").paging(0,3).dialect(2))
    results = r.ft(CACHE_INDEX).search(q, query_params={"vec": emb_bytes})
    for doc in results.docs:
        if (1 - float(doc.score)) >= SIMILARITY_THRESHOLD:
            return json.loads(doc.response)  # CACHE HIT
    return None  # CACHE MISS

def semantic_cache_set(prompt: str, response: dict, use_case: str = "default"):
    if _contains_pii(prompt):
        return  # NEVER cache PII
    h   = hashlib.sha256(prompt.encode()).hexdigest()[:16]
    key = f"semcache:{h}"
    r.json().set(key, "$", {
        "prompt":    prompt, "response": json.dumps(response),
        "use_case":  use_case, "created_at": int(time.time()),
        "embedding": _embed(prompt),
    })
    r.expire(key, TTL_MAP.get(use_case, TTL_MAP["default"]))
```

```python
def _contains_pii(text: str) -> bool:
    patterns = [r"[A-Z0-9._%+-]+@[A-Z0-9.-]+\.[A-Z]{2,}",
                r"\d{3}-\d{2}-\d{4}", r"Bearer [A-Za-z0-9\-._~+/]+=*"]
    return any(re.search(p, text, re.I) for p in patterns)
```

2b — Cache Warming at Deployment

PYTHON

```python
# cache/warming.py  — run as post-deploy job
import asyncio
from litellm import completion
from semantic_cache import semantic_cache_get, semantic_cache_set

# Load from query analytics — refresh weekly
HIGH_FREQ_QUERIES = [
    ("How do I write a unit test in pytest?", "test_generation"),
    ("Explain async/await in Python", "syntax_qa"),
    ("What is the difference between list and tuple?", "syntax_qa"),
    ("How does a transformer attention mechanism work?", "architecture"),
]

async def warm_cache(queries: list) -> dict:
    results = {"warmed": 0, "skipped": 0}
    for prompt, use_case in queries:
        if semantic_cache_get(prompt):
            results["skipped"] += 1; continue
        resp = completion(model="claude-haiku-4-5",
                          messages=[{"role":"user","content":prompt}],
                          max_tokens=512)
        semantic_cache_set(prompt, resp, use_case=use_case)
        results["warmed"] += 1
        await asyncio.sleep(0.1)   # rate-limit warming calls
    return results

if __name__ == "__main__":
    r = asyncio.run(warm_cache(HIGH_FREQ_QUERIES))
    print(f"Cache warmed: {r['warmed']} entries, skipped: {r['skipped']}")
```

3 LLM GATEWAY

Docker Compose Stack · PII Middleware · Audit Logging

The gateway is the single ingress point for all LLM traffic. Every request is tagged, PII-scrubbed, and written to an immutable audit log (Kafka → S3) before reaching any LLM provider. Required for EU AI Act Article 11 technical documentation compliance.

3a — Full Docker Compose Stack (docker-compose.yml)

YAML

```yaml
# docker-compose.yml  — full local dev stack
services:
  litellm-proxy:
    image: ghcr.io/berriai/litellm:main-latest
    ports: ["4000:4000"]
    volumes: ["./litellm_config.yaml:/app/config.yaml"]
    environment:
      ANTHROPIC_API_KEY:  ${ANTHROPIC_API_KEY}
      OPENAI_API_KEY:     ${OPENAI_API_KEY}
      DATABASE_URL:       postgresql://postgres:secret@postgres:5432/litellm
      LITELLM_MASTER_KEY: ${GATEWAY_MASTER_KEY}
      REDIS_HOST:         redis-cache
    depends_on: [postgres, redis-cache]

  router-service:
    build: ./router
    ports: ["8080:8080"]
    environment:
      LITELLM_URL:       http://litellm-proxy:4000
      REDIS_URL:         redis://redis-cache:6379
      ANTHROPIC_API_KEY: ${ANTHROPIC_API_KEY}
    depends_on: [litellm-proxy, redis-cache]

  redis-cache:
    image: redis/redis-stack:latest  # RediSearch + Vector
    ports: ["6379:6379", "8001:8001"]
    volumes: [redis_data:/data]

  langfuse:
    image: langfuse/langfuse:latest
    ports: ["3000:3000"]
    environment:
      DATABASE_URL: postgresql://postgres:secret@postgres:5432/langfuse
    depends_on: [postgres]

  prometheus:
    image: prom/prometheus:latest
    ports: ["9090:9090"]
    volumes: ["./prometheus.yml:/etc/prometheus/prometheus.yml"]

  grafana:
    image: grafana/grafana:latest
    ports: ["3001:3000"]
    volumes: [grafana_data:/var/lib/grafana]

  postgres:
    image: postgres:16
    environment:
      POSTGRES_PASSWORD: secret
    volumes: [postgres_data:/var/lib/postgresql/data]

volumes:
  redis_data: {}
  postgres_data: {}
  grafana_data: {}
```

3b — PII + Tagging + Audit Middleware (middleware.py)

```python
# gateway/middleware.py
from fastapi import Request, Response
from starlette.middleware.base import BaseHTTPMiddleware
import json, time, uuid, hashlib, asyncio, re, copy
import structlog, aiokafka

log = structlog.get_logger()

class AIGatewayMiddleware(BaseHTTPMiddleware):
    def __init__(self, app, kafka_producer):
        super().__init__(app)
        self.kafka = kafka_producer

    async def dispatch(self, request: Request, call_next):
        request_id = str(uuid.uuid4())
        start_ns   = time.perf_counter_ns()

        body = json.loads(await request.body() or b"{}")
        team_id    = request.headers.get("X-Team-ID",    "unknown")
        feature_id = request.headers.get("X-Feature-ID", "unknown")
        use_case   = request.headers.get("X-Use-Case",   "general")

        # PII scrub before any LLM call
        body, pii_found = scrub_pii(body)
        request._body   = json.dumps(body).encode()

        response  = await call_next(request)
        resp_body = b"".join([chunk async for chunk in response.body_iterator])

        try:
            resp_json  = json.loads(resp_body)
            usage      = resp_json.get("usage", {})
            input_tok  = usage.get("prompt_tokens",     0)
            output_tok = usage.get("completion_tokens", 0)
            model_used = resp_json.get("model", "unknown")
        except Exception:
            input_tok = output_tok = 0; model_used = "unknown"

        # Immutable audit entry -> Kafka -> S3 (EU AI Act Article 11)
        entry = {
            "request_id": request_id, "timestamp": time.time(),
            "team_id": team_id, "feature_id": feature_id,
            "use_case": use_case, "model": model_used,
            "input_tokens": input_tok, "output_tokens": output_tok,
            "latency_ms": (time.perf_counter_ns()-start_ns)//1_000_000,
            "prompt_hash": hashlib.sha256(
                json.dumps(body.get("messages",[])).encode()).hexdigest(),
            "pii_scrubbed": pii_found,
        }
        asyncio.create_task(
            self.kafka.send_and_wait("ai-audit-log", json.dumps(entry).encode())
        )
        return Response(content=resp_body, status_code=response.status_code,
                        headers=dict(response.headers), media_type=response.media_type)

PII_PATTERNS = {
    r"[A-Z0-9._%+-]+@[A-Z0-9.-]+\.[A-Z]{2,}"  : "[EMAIL]",
    r"\d{3}-\d{2}-\d{4}"                      : "[SSN]",
    r"Bearer [A-Za-z0-9\-._~+/]+=*"           : "Bearer [TOKEN]",
}

def scrub_pii(body: dict) -> tuple[dict, bool]:
```

```python
    text = json.dumps(copy.deepcopy(body)); found = False
    for pattern, repl in PII_PATTERNS.items():
        new_text, n = re.subn(pattern, repl, text, flags=re.I)
        if n: found = True
        text = new_text
    return json.loads(text), found
```

4 TOKEN BUDGET MANAGER

Hierarchical Enforcement · Circuit Breakers · Slack Alerts

Five-level budget hierarchy: Enterprise → BU → Team → Feature → Session. Redis atomic counters for real-time checks. At 80% consumed: Slack alert. At 95%: auto-downgrade model tier. At 100%: queue or reject with explanation.

Budget Threshold Actions

% Budget Remaining	Action	User Impact	Notification
> 20%	Proceed normally	None	None
10–20%	Proceed + soft alert	None visible	Slack to team lead
5–10%	Proceed + alert, prepare downgrade	Possible slower responses	Slack escalation
0–5%	Auto-downgrade model tier	Lower quality responses	Slack critical alert
0%	Queue non-urgent / reject urgent	Blocked until next period	Slack + email to manager

4a — Budget Manager (budget/manager.py)

PYTHON

```python
# budget/manager.py
import redis, json, time
from datetime import datetime
from dataclasses import dataclass
from enum import Enum

r = redis.Redis(host="redis-cache.internal", port=6379, decode_responses=True)

class BudgetAction(Enum):
    PROCEED    = "proceed"
    DOWNGRADE  = "downgrade"    # switch to cheaper model
    QUEUE      = "queue"        # non-urgent: defer to next period
    REJECT     = "reject"       # budget exhausted

BUDGETS = {  # monthly token allocations
    "teams": {
        "platform-team":  50_000_000,
        "data-team":      30_000_000,
        "frontend-team":  10_000_000,
    },
    "features": {
        "ai-assistant":   5_000_000,
        "code-review":    3_000_000,
        "docs-generator": 2_000_000,
    },
}

DOWNGRADE_CHAIN = {
    "claude-opus-4-6":   "claude-sonnet-4-6",
    "claude-sonnet-4-6": "claude-haiku-4-5",
    "claude-haiku-4-5":  None,  # floor
}

def get_budget_status(team_id: str, feature_id: str,
                      current_model: str = "claude-sonnet-4-6") -> dict:
    period = datetime.now().strftime("%Y-%m")
    most_restrictive = 1.0

    for scope, budget in [
        (f"team:{team_id}",       BUDGETS["teams"].get(team_id, 10_000_000)),
        (f"feature:{feature_id}", BUDGETS["features"].get(feature_id, 1_000_000)),
    ]:
        used = int(r.get(f"budget:{scope}:{period}") or 0)
        pct  = max(0, budget - used) / budget
        if pct < most_restrictive:
            most_restrictive = pct

    if most_restrictive == 0:
        return {"allowed": False, "action": BudgetAction.REJECT,
                "remaining_pct": 0, "downgrade_to": None}

    if most_restrictive < 0.05:
        downgrade = DOWNGRADE_CHAIN.get(current_model)
        if downgrade:
            return {"allowed": True, "action": BudgetAction.DOWNGRADE,
                    "remaining_pct": most_restrictive, "downgrade_to": downgrade}
        return {"allowed": False, "action": BudgetAction.QUEUE,
                "remaining_pct": most_restrictive, "downgrade_to": None}

    if most_restrictive < 0.20:
        _trigger_alert(team_id, feature_id, most_restrictive)
```

PYTHON

```python
    return {"allowed": True, "action": BudgetAction.PROCEED,
            "remaining_pct": most_restrictive, "downgrade_to": None}

def record_usage(team_id: str, feature_id: str, tokens: int, model: str):
    period = datetime.now().strftime("%Y-%m")
    pipe = r.pipeline()
    for scope in [f"team:{team_id}", f"feature:{feature_id}", "enterprise"]:
        key = f"budget:{scope}:{period}"
        pipe.incrby(key, tokens)
        pipe.expireat(key, _next_month_ts())
    pipe.execute()

def _trigger_alert(team_id, feature_id, pct):
    alert_key = f"alert_sent:{team_id}:{feature_id}:{datetime.now().strftime('%Y-%m-%d')}"
    if not r.get(alert_key):
        r.publish("budget-alerts", json.dumps({
            "team_id": team_id, "feature_id": feature_id,
            "remaining": round(pct * 100, 1), "timestamp": time.time(),
        }))
        r.setex(alert_key, 86400, "1")

def _next_month_ts() -> int:
    now = datetime.now()
    yr  = now.year + (1 if now.month == 12 else 0)
    mo  = 1 if now.month == 12 else now.month + 2
    return int(datetime(yr, mo, 1).timestamp())
```

5 AI FINOPS

OpenTelemetry Metrics · Cost Attribution · Grafana Dashboard

Tag every LLM call with team_id, feature_id, use_case, model, and environment. Emit Prometheus metrics via OpenTelemetry. Store cost in micro-dollars (integer) to avoid float drift at scale. Grafana dashboards expose cost by team, model tier distribution, cache hit rate, and anomaly detection (2x rolling average alert).

5a — OpenTelemetry Metrics (observability/metrics.py)

PYTHON

```python
# observability/metrics.py
from opentelemetry import trace, metrics
from opentelemetry.sdk.trace import TracerProvider
from opentelemetry.sdk.metrics import MeterProvider
from opentelemetry.exporter.otlp.proto.grpc.trace_exporter import OTLPSpanExporter
from opentelemetry.exporter.prometheus import PrometheusMetricReader
from prometheus_client import start_http_server

tracer_provider = TracerProvider()
tracer_provider.add_span_exporter(OTLPSpanExporter(endpoint="http://jaeger.internal:4317"))
trace.set_tracer_provider(tracer_provider)

reader = PrometheusMetricReader()
metrics.set_meter_provider(MeterProvider(metric_readers=[reader]))
start_http_server(8888)   # Prometheus scrapes :8888/metrics

tracer = trace.get_tracer("ai-gateway")
meter = metrics.get_meter("ai-gateway")

token_counter = meter.create_counter("llm_tokens_total", unit="tokens")
cost_counter = meter.create_counter("llm_cost_usd_total", unit="microdollars")
latency_histogram = meter.create_histogram("llm_request_latency_ms", unit="ms")
cache_hit_counter = meter.create_counter("llm_cache_hits_total")

MODEL_COSTS = {
    "claude-haiku-4-5": {"input": 0.00025, "output": 0.00125},
    "claude-sonnet-4-6": {"input": 0.003, "output": 0.015},
    "claude-opus-4-6": {"input": 0.015, "output": 0.075},
    "gpt-4o-mini": {"input": 0.00015, "output": 0.0006},
}

def record_llm_call(team_id, feature_id, use_case, model,
                    input_tokens, output_tokens, latency_ms, cached=False):
    tier = ("nano" if "haiku" in model or "flash" in model or "mini" in model
            else "mid" if "sonnet" in model or "4o" in model else "frontier")

    labels = dict(team_id=team_id, feature_id=feature_id,
                  use_case=use_case, model=model, tier=tier)

    token_counter.add(input_tokens, {**labels, "token_type": "input"})
    token_counter.add(output_tokens, {**labels, "token_type": "output"})

    costs = MODEL_COSTS.get(model, {"input": 0.003, "output": 0.015})
    cost_usd = (input_tokens / 1000 * costs["input"] +
                output_tokens / 1000 * costs["output"])
    cost_counter.add(int(cost_usd * 1_000_000), labels)  # store as microdollars
    latency_histogram.record(latency_ms, labels)

    if cached:
        cache_hit_counter.add(1, {"team_id": team_id, "use_case": use_case})

    with tracer.start_as_current_span("llm_call") as span:
        span.set_attributes({"llm.model": model, "llm.cost_usd": round(cost_usd, 6),
                             "llm.cached": cached, "team.id": team_id,
                             "llm.input_tokens": input_tokens,
                             "llm.output_tokens": output_tokens})
```

5b — Key Grafana Prometheus Queries

Panel	PromQL Expression	Purpose
Monthly Spend (USD)	sum(increase(llm_cost_usd_total[30d])) / 1000000	Total AI spend this month

Panel	PromQL Expression	Purpose
Cost by Team (7d)	sum by(team_id)(increase(llm_cost_usd_total[7d]))/1000000	Chargeback view per team
Cache Hit Rate	rate(llm_cache_hits_total[5m]) / rate(llm_tokens_total[5m]) * 100	Semantic cache effectiveness
Model Tier Dist.	sum by(tier)(increase(llm_tokens_total[24h]))	Nano/Mid/Frontier split
Cost Anomaly	rate(llm_cost_usd_total[5m]) > avg_over_time(rate(llm_cost_usd_total[5m])[7d:5m]) * 2	2x rolling average breach alert
P95 Latency by Model	histogram_quantile(0.95, sum by(model,le)(rate(llm_request_latency_ms_bucket[5m])))	Latency per model tier

6 ROI MEASUREMENT

DORA + SPACE + AI FinOps Unified Scorecard

Key Benchmarks (2026): Healthy ROI = 2.5–3.5x average, 4–6x top quartile. AI code churn baseline 3.3% (pre-AI) → healthy post-AI <5.7% → critical >7.1%. Year-1 ROI is almost always negative due to integration costs. True signal emerges year 2–3.

ROI Scorecard — Health Thresholds

Metric	Collection	Green	Yellow	Red
Lead time commit→prod	Git analytics	< 1 week	1–4 weeks	> 4 weeks
Deploy frequency	CI/CD pipeline	Daily+	Weekly	Monthly
Change failure rate	Incident tracking	< 5%	5–15%	> 15%
MTTR	Incident tracking	< 1 hour	1–24 hours	> 24 hours
AI suggestion acceptance	Tool analytics	> 30%	15–30%	< 15%
30-day AI code churn	GitClear / git analysis	< 5.7%	5.7–7.1%	> 7.1%
Security findings delta	SAST (Semgrep, Snyk)	< baseline	1.5x baseline	> 2x baseline
Cost per merged AI PR	LLM gateway + git	< $2.00	$2–$10	> $10
Cache hit rate	Gateway metrics	> 30%	20–30%	< 20%
Frontier model usage %	Routing logs	< 10%	10–15%	> 15%

6 — ROI Calculator (roi/scorecard.py)

PYTHON

```python
# roi/scorecard.py
from dataclasses import dataclass

@dataclass
class DORAsignals:
    lead_time_days: float; deploy_freq_week: float
    change_failure_rate: float; mttr_hours: float

@dataclass
class AIFinOpsSignals:
    cost_per_pr_usd: float; token_efficiency_ratio: float
    cache_hit_rate: float; monthly_spend_usd: float
    model_tier_dist: dict   # {nano: 0.82, mid: 0.16, frontier: 0.02}

def calculate_roi(dora: DORAsignals, code_churn: float,
                  finops: AIFinOpsSignals, team_size: int = 10,
                  avg_dev_hourly: float = 100.0) -> dict:
    alerts = []

    # Productivity value via DORA lead time reduction
    baseline_lt      = 14.0   # days (pre-AI industry benchmark)
    lt_reduction     = max(0, (baseline_lt - dora.lead_time_days) / baseline_lt)
    hrs_saved_yr     = lt_reduction * 40 * 52 * 0.15   # 15% of work hours
    productivity_val = hrs_saved_yr * avg_dev_hourly * team_size

    # Quality adjustment
    quality_adj = 0.0
    if code_churn > 0.071:
        alerts.append(f"Code churn {code_churn:.1%} exceeds 7.1% — review AI code quality")
        quality_adj = -0.2
    if dora.change_failure_rate > 0.15:
        alerts.append(f"Change failure rate critical: {dora.change_failure_rate:.1%}")

    # FinOps alerts
    if finops.cost_per_pr_usd > 10:
        alerts.append(f"Cost/PR ${finops.cost_per_pr_usd:.2f} exceeds $10 threshold")
    if finops.cache_hit_rate < 0.30:
        alerts.append(f"Cache hit rate {finops.cache_hit_rate:.1%} below 30%")
    if finops.model_tier_dist.get("frontier", 0) > 0.15:
        alerts.append(f"Frontier usage {finops.model_tier_dist['frontier']:.1%} > 15%")

    annual_cost = finops.monthly_spend_usd * 12
    roi_mult    = productivity_val * (1 + quality_adj) / annual_cost if annual_cost else 0
    status      = "green" if roi_mult >= 2.5 and not alerts else "yellow" if ro..

    return {"roi_multiplier": round(roi_mult, 2),
            "status": status, "alerts": alerts,
            "productivity_value_usd": round(productivity_val),
            "annual_ai_cost_usd": round(annual_cost)}
```

7 AGENTIC COST CONTROL

Budget Guard · Lazy Tool Loading · Session Limits

Critical Risk: Claude Opus 4.7 at 27x multiplier on Copilot annual plans. A stuck 2-hour agentic loop can consume $500+ of budget in minutes. Hard token limits and step caps are MANDATORY — never optional.

7a — Budget-Guarded Agent Wrapper (agents/budget_guard.py)

PYTHON

```python
# agents/budget_guard.py
from dataclasses import dataclass, field
from typing import Callable
import time, asyncio, structlog

log = structlog.get_logger()

@dataclass
class AgentBudget:
    max_tokens:    int   = 50_000   # total session token limit
    max_steps:     int   = 20       # maximum loop iterations
    max_cost_usd: float  = 5.00     # hard dollar ceiling
    warn_at_pct:   float  = 0.80     # warn when 80% consumed

@dataclass
class AgentBudgetTracker:
    budget: AgentBudget
    tokens_used:     int   = 0
    steps_completed: int   = 0
    cost_usd:        float = 0.0
    warnings_sent:   list  = field(default_factory=list)
    start_time:      float = field(default_factory=time.time)

    MODEL_COSTS = {
        "claude-haiku-4-5":  0.000875,
        "claude-sonnet-4-6": 0.009,
        "claude-opus-4-6":   0.045,
    }

    def record_step(self, tokens: int, model: str):
        self.tokens_used     += tokens
        self.steps_completed += 1
        self.cost_usd        += tokens / 1000 * self.MODEL_COSTS.get(model, 0.009)

    def should_stop(self) -> tuple[bool, str]:
        if self.steps_completed >= self.budget.max_steps:
            return True, f"step_limit ({self.budget.max_steps})"
        if self.tokens_used >= self.budget.max_tokens:
            return True, f"token_limit ({self.tokens_used:,})"
        if self.cost_usd >= self.budget.max_cost_usd:
            return True, f"cost_limit (${self.cost_usd:.3f})"
        return False, ""

class BudgetGuardedAgent:
    def __init__(self, agent_fn: Callable, budget: AgentBudget,
                 team_id: str, session_id: str):
        self.agent_fn  = agent_fn
        self.tracker   = AgentBudgetTracker(budget=budget)
        self.team_id   = team_id
        self.session_id = session_id

    async def run(self, initial_state: dict) -> dict:
        state = initial_state
        log.info("agent_start", session=self.session_id,
                 budget_tokens=self.tracker.budget.max_tokens,
                 budget_usd=self.tracker.budget.max_cost_usd)

        while True:
            should_stop, reason = self.tracker.should_stop()
            if should_stop:
                log.warning("agent_budget_stop", session=self.session_id,
                            reason=reason, tokens=self.tracker.tokens_used,
```

```python
                            cost=round(self.tracker.cost_usd, 4))
                return {**state, "__budget_stop__": reason, "__final__": True}

            result = await self.agent_fn(state)
            self.tracker.record_step(result.get("__tokens__", 0),
                                     result.get("__model__", "claude-sonnet-4-6"))
            state = result
            if state.get("__final__") or state.get("__done__"):
                break

        log.info("agent_end", session=self.session_id,
                 total_tokens=self.tracker.tokens_used,
                 total_cost=round(self.tracker.cost_usd, 4),
                 steps=self.tracker.steps_completed)
        return state
```

7b — Lazy Tool Loading (agents/lazy_tools.py)

Anti-pattern AP-03 fix. 20 tools x 200 tokens = 4,000 tokens/step wasted. Lazy loading selects 3–5 relevant tools per step. 15-step session savings: (4,000 - 800) × 15 = 48,000 tokens ~ $0.72 per session at Opus rates.

```python
# agents/lazy_tools.py
from sentence_transformers import SentenceTransformer, util
from dataclasses import dataclass
import torch

encoder = SentenceTransformer("all-MiniLM-L6-v2")

@dataclass
class Tool:
    name: str; description: str
    schema: dict; token_cost: int   # approx tokens in schema

# Full tool registry (20+ tools in production)
TOOLS = [
    Tool("read_file",    "Read file contents from disk",    {}, 180),
    Tool("write_file",   "Write content to a file",         {}, 195),
    Tool("run_tests",    "Execute pytest test suite",       {}, 220),
    Tool("search_web",   "Search the web for information",  {}, 210),
    Tool("query_db",     "Execute SQL query against database", {}, 280),
    Tool("create_pr",    "Create a GitHub pull request",    {}, 310),
    Tool("execute_code","Run Python code in sandbox",      {}, 200),
    Tool("list_files",   "List directory contents",         {}, 150),
    # ... 12+ more tools
]

# Pre-compute embeddings at startup (once)
_tool_embs = encoder.encode([t.description for t in TOOLS], convert_to_tensor=True)

def select_tools(step_description: str, top_k: int = 4,
                 max_tokens: int = 800) -> list[Tool]:
    """Select top_k relevant tools within token budget."""
    step_emb = encoder.encode(step_description, convert_to_tensor=True)
    scores   = util.cos_sim(step_emb, _tool_embs)[0]
    ranked   = torch.argsort(scores, descending=True).tolist()

    selected = []; tokens_used = 0
    for idx in ranked[:top_k * 2]:
        tool = TOOLS[idx]
        if tokens_used + tool.token_cost <= max_tokens:
            selected.append(tool); tokens_used += tool.token_cost
        if len(selected) >= top_k: break
    return selected

# In agent loop:
# tools = select_tools(current_step_description)    # 3-5 tools
# schemas = [{"type":"function","function":{"name":t.name,"description":t.description,
#             "parameters":t.schema or {}}} for t in tools]
# response = llm_client.chat(messages=..., tools=schemas)
```

8 ANTI-PATTERN FIXES

History Compression · Prompt Trimming · Batch Routing

8a — Conversation History Compression (AP-02 Fix)

A 50-turn conversation adds 30K+ tokens to every subsequent call. Compress after N turns using a cheap Haiku summarization call ($0.0002). Saves $0.05+ per subsequent call — a 250:1 ROI on the compression cost.

PYTHON

```python
# compression/history.py
from litellm import completion
import json

SUMMARY_PROMPT = """Compress this conversation history into a concise JSON summary.
Preserve: key decisions, files/modules discussed, open questions.
Output ONLY valid JSON: {"decisions":[],"context":"","open_items":[],"turn_count":0}"""

def compress_history(messages: list[dict], compress_after: int = 10,
                     keep_recent: int = 3) -> list[dict]:
    """
    Compress old turns into a summary after N turns.
    Reduction: 60-80% of conversation history tokens.
    Cost to compress: ~$0.0002 (Haiku). Savings per call: $0.05+
    """
    turn_count = sum(1 for m in messages if m["role"] == "user")
    if turn_count <= compress_after:
        return messages    # nothing to compress yet

    system_msgs = [m for m in messages if m["role"] == "system"]
    convo_msgs  = [m for m in messages if m["role"] != "system"]

    if len(convo_msgs) <= keep_recent * 2:
        return messages

    old_msgs    = convo_msgs[:-(keep_recent * 2)]
    recent_msgs = convo_msgs[-(keep_recent * 2):]

    summary_resp = completion(
        model="claude-haiku-4-5",    # always use cheapest for summarization
        messages=[{"role": "system", "content": SUMMARY_PROMPT},
                  {"role": "user",   "content": json.dumps(old_msgs)}],
        max_tokens=300,
    )
    try:
        summary = json.loads(summary_resp.choices[0].message.content)
    except (json.JSONDecodeError, KeyError):
        return messages    # fail safely — return original if summary fails

    summary_msg = {
        "role":    "system",
        "content": (
            f"[COMPRESSED HISTORY — {summary.get('turn_count',0)} turns]
"
            f"Decisions: {'; '.join(summary.get('decisions', []))}
"
            f"Context:   {summary.get('context', '')}
"
            f"Open:      {'; '.join(summary.get('open_items', []))}"
        )
    }
    return system_msgs + [summary_msg] + recent_msgs
```

8b — System Prompt Compression (AP-01 Fix)

PYTHON

```python
# compression/prompt.py
def compress_system_prompt (prompt : str) -> str:
    """
    Remove filler phrases and deduplicate similar lines.
    Quick heuristic — use LLMLingua library for production-grade compression.
    Target: 30-50% reduction with <5% quality loss.
    """
    FILLER_PHRASES = [
        "please note that", "it is important to", "make sure to",
        "always remember", "keep in mind that", "as an ai",
        "i want you to", "you should always", "please ensure that",
    ]
    lines      = prompt.strip().split("\n")
    filtered   = []
    seen_keys  = set()

    for line in lines:
        stripped = line.strip()
        if not stripped:
            continue
        # Remove filler
        if any(f in stripped.lower() for f in FILLER_PHRASES):
            continue
        # Deduplicate semantically similar lines (first 5 words as key)
        key = frozenset(stripped.lower().split()[:5])
        if key in seen_keys:
            continue
        seen_keys.add(key)
        filtered.append(line)

    original_tok   = len(prompt.split())
    compressed     = "\n".join(filtered)
    compressed_tok = len(compressed.split())
    print(f"Prompt: {original_tok} -> {compressed_tok} tokens "
          f"({(1 - compressed_tok/original_tok):.1%} reduction)")
    return compressed

# ■■ Batch API Router (AP-14 Fix) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
async def route_to_batch_if_eligible (request: dict) -> dict:
    """
    Non-urgent workloads qualify for 50% batch API discount.
    Latency tolerance > 30 seconds → use batch pricing.
    """
    BATCH_ELIGIBLE_USE_CASES = {
        "docs_generation", "test_generation", "code_quality_scan",
        "dependency_audit", "changelog_generation", "migration_analysis",
    }
    use_case = request.get("use_case", "")
    if use_case in BATCH_ELIGIBLE_USE_CASES:
        return {**request, "use_batch_api": True,
                "estimated_discount": "50%",
                "expected_latency": "15-30 minutes"}
    return {**request, "use_batch_api": False}
```

9 ARCHITECTURE INTEGRATION MAP

How All Components Connect

The Enterprise AI Cost Control Platform (EACCP) connects all modules through a single gateway. No consuming application changes are needed — the gateway handles routing, caching, budgeting, and observability transparently.

Request Flow (Every LLM Call)

Step	Component	File	Action	Cost Impact
1	Consumer App	Any app/agent	Call POST /v1/chat/completions with headers: X-Team-ID, X-Feature-ID, X-Use-Case	N/A
2	Gateway Middleware	middleware.py	PII scrub + tag injection + audit log write	~0ms; $0
3	Semantic Cache	semantic_cache.py	Embed query, cosine similarity check vs Redis cache	$0.00002 embed $0 if HIT (30–60% rate)
4	Complexity Classifier	classifier.py	Score complexity 0.0–1.0 using SBERT	~10ms; $0.00001
5	Budget Check	budget/manager.py	Check team + feature budget remaining pct	<1ms; $0
6	Model Router	router/main.py	Select nano/mid/frontier based on score + budget	$0
7	LiteLLM Proxy	litellm_config.yaml	Provider-agnostic LLM call with fallback chain	Actual LLM cost
8	Metrics Emit	observability/metrics.py	Record tokens, cost (microdollars), latency to Prometheus	<1ms; $0
9	Budget Record	budget/manager.py	Atomic increment of team + feature usage counters	<1ms; $0
10	Cache Store	semantic_cache.py	Write response + embedding to Redis with TTL	$0.00002 embed

Component Dependency Matrix

Component	Depends On	Depended On By	Failure Mode	Fallback
Semantic Cache (Redis)	Redis Stack	Router, LiteLLM proxy	Cache miss → normal LLM call	Fail open; bypass cache
Complexity Classifier	SBERT model file (local)	Router engine	Default to Mid tier	Use Mid model for all
Budget Manager	Redis (counters)	Router engine	Allow-with-alert mode	Permit all at base tier
LiteLLM Proxy	Provider API keys	Router engine, all consumers	Fallback to alternate provider	Queue + retry x3
Langfuse	Postgres	LiteLLM (callback)	Non-blocking; buffer locally	Write to local log file
Kafka (audit)	Kafka cluster	Gateway middleware	Buffer in-memory; batch flush	Write to Postgres audit table

10 QUICK-START DEPLOYMENT CHECKLIST

Week-by-Week Implementation Sequence

Week 1 — Foundation (Highest ROI)

- ■ Deploy RedisStack (Docker): redis/redis-stack:latest → port 6379 + 8001
- ■ Deploy LiteLLM Proxy with litellm_config.yaml defining nano/mid/frontier tiers
- ■ Add X-Team-ID, X-Feature-ID, X-Use-Case headers to all existing LLM calls
- ■ Enable Langfuse callback in LiteLLM → instant cost visibility dashboard
- ■ Set up basic Prometheus scraping on port 8888 → Grafana dashboard import
- ■ Expected outcome: Full cost visibility. Baseline established for optimization.

Week 2 — Caching (30–60% Cost Reduction)

- ■ Implement semantic_cache.py with Redis Vector Search index (HNSW)
- ■ Add cache_get check before every LLM call in router/main.py
- ■ Add cache_set after every LLM response (with PII check)
- ■ Configure TTL map by use_case (30 days for syntax, 1 hour for security)
- ■ Run cache warming script on top-100 FAQ queries from Langfuse analytics
- ■ Target: >30% cache hit rate within 72 hours of deployment.

Week 3 — Routing (60–87% Additional Savings)

- ■ Deploy classifier.py with SBERT model (pip install sentence-transformers)
- ■ Integrate classifier into routing engine: score → model selection
- ■ A/B test: route 10% traffic to new router, compare quality vs full-Frontier baseline
- ■ Validate: Nano-tier suggestion acceptance rate > 25% (acceptable quality)
- ■ Gradually increase routing percentage to 100% as quality validates
- ■ Expected outcome: 60–87% cost reduction vs always-Frontier baseline.

Week 4 — Budget Governance

- ■ Define monthly token budgets per team and feature in BUDGETS dict
- ■ Deploy budget manager with Redis atomic counters
- ■ Configure Slack webhook for budget alert notifications
- ■ Set budget alert thresholds: 80% (soft) / 95% (downgrade) / 100% (queue)
- ■ Add budget check step to routing engine before LLM dispatch
- ■ Brief all team leads on budget dashboard URL and interpretation.

Month 2 — Agentic Controls + ROI Measurement

- ■ Wrap all agent loops with BudgetGuardedAgent (max_steps=20, max_cost=$5)
- ■ Implement lazy tool loading (select_tools) in all agent workflows
- ■ Implement history compression (compress after 10 turns, keep last 3)
- ■ Run 90-day ROI pilot: measure DORA metrics before vs after
- ■ Set up GitClear or equivalent for code churn tracking
- ■ Monthly FinOps review: team cost reports + efficiency ratio ranking

pip install dependencies

PYTHON

```
# requirements.txt  — complete dependency set
litellm>=1.40.0
sentence-transformers>=2.7.0
redis[hiredis]>=5.0.0
fastapi>=0.110.0
uvicorn[standard]>=0.29.0
pydantic>=2.0.0
opentelemetry-sdk>=1.24.0
opentelemetry-exporter-otlp-proto-grpc>=1.24.0
opentelemetry-exporter-prometheus>=0.45b0
prometheus-client>=0.20.0
structlog>=24.0.0
aiokafka>=0.10.0
httpx>=0.27.0
numpy>=1.26.0
openai>=1.30.0          # for embedding model (text-embedding-3-small)
langfuse>=2.0.0         # LLM observability (optional but recommended)

# Run services:
# docker compose up -d                        # full stack
# uvicorn gateway.main:app --port 8080        # router service
# python cache/warming.py                     # seed cache
# python budget/alert_consumer.py             # slack alerts
```